

# Torch

`torch.index_select()`

`torch.tensor.detach().cpu().numpy()`

- `device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')`

gradient clipping simple solution to gradient explosion, works well practically

- `torch.nn.utils.clipgrad_norm(parameters, max_norm, norm_type=2)`
  - example: `torch.nn.utils.clipgrad_norm(retinanet.parameters(), 0.1)`

## 1. Chapter 2

```
1. import torch
   device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")

   net.to(device)
   images = images.to(device)
   labels = labels.to(device)
   output = net(images)
   loss = criterion(output, labels)
```

## Pytorch

### Tensor

1. From an interface perspective, operations on Tensors can be divided into two categories:
  1. `torch.function`, such as `torch.save`, etc.
  2. `tensor.function`, such as `tensor.view`, etc.
2. For ease of use, most operations on Tensors support both types of interfaces simultaneously, and this book does not make a specific distinction, for example, `torch.sum(a, b)` and `a.sum(b)` are functionally equivalent.
3. From a storage perspective, operations on Tensors can be divided into two categories:
  1. A function that does not modify its own data, such as `a.add(b)` will return a new Tensor as the result of the addition.
  2. A function that modifies its own data, such as `a.add_(b)` will still store the result if the addition in `a`, but `a` will be modified.
4. Functions whose names end with an underscore are inplace functions, meaning they modify the caller's own data. This distinction is important in practical applications.

### Creating Tensors

1. There are many ways to create tensors in Pytorch, as shown in Table 3-1.

| Function                                          | Functionality                                                                 |
|---------------------------------------------------|-------------------------------------------------------------------------------|
| <code>Tensor(*sizes)</code>                       | Basic constructor                                                             |
| <code>tensor(data,)</code>                        | Constructor similar to <code>np.array()</code>                                |
| <code>ones(*sizes)</code>                         | Tensor with all ones                                                          |
| <code>zeros(*sizes)</code>                        | Tensor with all zeros                                                         |
| <code>eye(*sizes)</code>                          | The diagonal is 1, and the rest are 0                                         |
| <code>arange(s, e, step)</code>                   | From <code>s</code> to <code>e</code> , the step size is <code>step</code>    |
| <code>linspace(s, e, steps)</code>                | From <code>s</code> to <code>e</code> , divide evenly into <code>steps</code> |
| <code>rand/randn(*sizes)</code>                   | uniform/standard distribution                                                 |
| <code>normal(means, std)/uniform(from, to)</code> | normal distribution/ uniform distribution                                     |
| <code>randperm(m)</code>                          | random arrangement                                                            |

| Function                               | Functionality                                                                                                                                                                                |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>tensor.new_*/torch.*_like</code> | create a Tensor of the same size, filled with * type (e.g. <code>tensor_new_ones</code> , is filled with all ones), and with the same <code>torch.dtype</code> and <code>torch.device</code> |

- All these creation methods allow you to specify the data type (dtype) and the storage device (CPU/GPU) during creation.
- The tensor function can be used to create a new Tensor. It can accept a list and create a new tensor based on the data in the list. It can also create a tensor based on the data in the list. It can also create a tensor based on a specified data shape and can pass in other tensors. Several examples will be given below.
- It's important to note that when `t.Tensor(*sizes)` creates a Tensor, the system doesn't immediately allocate space. It only calculates whether there's enough remaining memory and allocates space immediately after tensor creation. In practice we recommend using `torch.tensor` instead of `torch.Tensor`. Let's compare the differences between the two.
- `torch.Tensor` is a python class, and by default it is `torch.FloatTensor()`. Running `torch.Tensor([2, 3])` will directly call the `__init__()` constructor of the `Tensor` class, generating a single-precision floating-point `Tensor(FloatTensor)`.
- `torch.tensor` is a python function with the prototype `torch.tensor(data, dtype=None, device=None, requires_grad=False)`. `data` supports types such as list, tuple, array, and scalar. `torch.tensor()` directly copies data from `data` and generates the corresponding Tensor based on the original data type.
- Since `torch.tensor()` can generate a corresponding Tensor based on the data type, we recommend using the `torch.tensor()` method to create a new Tensor in practical applications.

## Types of Tensor

- Tensor types can be further divided into device type and data type (dtype). Device type is divided into CUDA and CPU types, and numeric types include bool, float, and int.
- Tensor data types are shown in Table 3-2, and each data type has CPU and GPU versions. Readers can obtain the device type of a tensor using `tensor.device` and the data type using `tensor.dtype`.
- The default data type of a Tensor is `FloatTensor`. Readers can modify the default Tensor type using `torch.set_default_tensor_type` (if the default type is a GPPU Tensor, then all operations will be performed on the GPU) or `torch.set_default_dtype` (only accepts float input types).
- Understanding the type of a Tensor is helpful for analyzing memory usage. For example, a `FloatTensor` of shape (1000, 1000, 1000) has  $1000 \times 1000 \times 1000 = 10^9$  elements, each occupy  $32\text{bit} \div 8 = 4\text{byte}$  of memory/GPU memory. `HalfTensor` is specifically designed for GPU versions. With the same number of elements, `HalfTensor` uses only half the GPU memory of a `FloatTensor`.
- Therefore, using `HalfTensor` can greatly alleviate the problem of insufficient GPU memory. It should be noted that `HalfTensor` has limited numerical values and precision, which may lead to data overflow issues.
- Tensor data type

| Data Type                            | dtype                                                   | CPU tensor                      | GPU tensor                           |
|--------------------------------------|---------------------------------------------------------|---------------------------------|--------------------------------------|
| 32-bit floating-point                | <code>torch.float32</code> or <code>torch.float</code>  | <code>torch.FloatTensor</code>  | <code>torch.cuda.FloatTensor</code>  |
| 64-bit floating-point                | <code>torch.float64</code> or <code>torch.double</code> | <code>torch.DoubleTensor</code> | <code>torch.cuda.DoubleTensor</code> |
| 16-bit half-precision floating-point | <code>torch.float16</code> or <code>torch.half</code>   | <code>torch.HalfTensor</code>   | <code>torch.cuda.HalfTensor</code>   |
| 8-bit unsigned integer               | <code>torch.uint8</code>                                | <code>torch.ByteTensor</code>   | <code>torch.cuda.ByteTensor</code>   |
| 8-bit signed integer                 | <code>torch.int8</code>                                 | <code>torch.CharTensor</code>   | <code>torch.cuda.CharTensor</code>   |
| 16-bit signed integer                | <code>torch.int16</code> or <code>torch.short</code>    | <code>torch.ShortTensor</code>  | <code>torch.cuda.ShortTensor</code>  |
| 32-bit signed integer                | <code>torch.int32</code> or <code>torch.int</code>      | <code>torch.IntTensor</code>    | <code>torch.cuda.IntTensor</code>    |
| 64-bit signed integer                | <code>torch.int64</code> or <code>torch.long</code>     | <code>torch.LongTensor</code>   | <code>torch.cuda.LongTensor</code>   |
| Boolean type                         | <code>torch.bool</code>                                 | <code>torch.BoolTensor</code>   | <code>torch.cuda.BoolTensor</code>   |

- Common methods for converting between different types of tensors are as follows:
  - The most common approach is `tensor.type(new_type)`, but there are also shortcut methods such as `tensor.float()`, `torch.long()`, and `tensor.half()`.
  - Conversion between CPU tensors and GPU tensors is achieved using `tensor.cuda()` and `tensor.cpu()`, and `tensor.to(device)` can also be used.
  - Creating tensors of the same type: `torch.*_like` and `torch.new_*`. These two methods are suitable for writing device compatible code:

1. `torch.*_like(tensorA)` generates a new tensor with the same attributes as `tensorA` (such as type, shape and CPU/GPU)
2. `tensor.new_*(new_shape)` creates a new tensor with a different shape but the same attributes.